面试高频问题清单（京东供应链前端）

> 用法：先看每题的**简短回答**，能讲 2 分钟再勾上「已掌握」。想深入时再展开详解。 【】里是只有你知道的真实细节，面试前必须换成自己的说法。技术题可以用自己的话讲，不必背原文。
>
> 原版已备份在同目录的 `面试高频问题清单-原版.md` 。

学习路线

每天 3～4 小时的话，第一档加第二档大约 7～10 天，足够去面试。第三档每个能说三五个要点即可。

档位	内容	时间	标准
第一档	简历上写了、一定会被追问的项目	4～5 天	每题讲 2 分钟，追问两三层不卡
第二档	Vue、JS、浏览器、工程化、手写	2～3 天	能讲清原理，手写题能写出来
第三档	A/B、大屏、BI、低代码、监控、AI 落地	1～2 天	说出要点，不必背代码

目录

第一档

- 开场
- 易鑫：微前端、工程化、AI
- 慧策：表格、上传、业务

- 权限、小程序、早期项目

第二档

- Vue / React
- JS / 浏览器 / 工程化 / 算法

第三档

- A/B、大屏、BI、低代码、监控、供应链

一、开场

1. 用 2 分钟介绍自己 ★★★

- [ ] 已掌握

简短回答：

我叫顾彬，计算机本科，9 年研发经验，主做 B 端 SaaS，也做过小程序和 Android。最近在易鑫做贷前后台，

Monorepo 加京东 MicroApp，29 个子应用。我做了多环境构建和分包，主包从 5MB 降到 2MB；改了发布流程，拦住两次错误合并；把 CRUD 页面沉淀成 Skill，同类页面从 4 天缩短到 1 天。之前在慧策 4 年，做慧经营 SaaS，对接旺店通 ERP 和京东、抖音、淘宝的订单数据。我负责的三个分析模块占销售额 50% 以上，做过大数据表格、配置化报表、大文件上传，也带过小组。更早做过采购、入库、出库这类单据系统。供应链 B 端的复杂表格、表单、权限和配置化，正好是我这些年在做的事。

2. 为什么从易鑫离职？只做了 10 个月 ★★★

- [ ] 已掌握

简短回答：

用客观原因，不抱怨公司。可以说业务调整，自己负责的部分收缩了，这 10 个月该交付的都交付了，线上 P0/P1 为 0。也可以说想回到电商和供应链方向，易鑫是金融贷前。说完立刻接到“所以京东供应链和我以前的经历匹配”。不要说加班多、钱少、领导不好。

展开：两种说法

- 业务调整：“【团队业务调整 / 项目方向变化】，我负责的部分【收缩 / 合并】了。任职期间需求按期交付，线上 P0/P1 问题 0 起。”
- 方向选择：“易鑫是金融贷前，我更想回到电商和供应链。之前在慧策做的就是电商 ERP 数据。”

3. 2025.7 到 2026.1 这半年在做什么 ★★★

- [ ] 已掌握

简短回答：

离开慧策后系统补了 AI 方向，学了 Python、FastAPI 和 Agent，【做了什么小项目】。这也是后来在易鑫能比较快落地 AI 提效的原因。如果还有家里的事，一句带过。不要含糊。

4. 为什么来京东、为什么是供应链 ★★

- [ ] 已掌握

简短回答：

业务上连续：慧经营对接过京东等平台的订单，也做过进销存单据。技术上匹配：供应链系统表格多、表单复杂、状

态多、权限细。平台上，京东供应链的规模值得深入，而且我最近用的就是京东开源的 MicroApp。

5. 期望薪资和到岗时间 ★★

- [ ] 已掌握

简短回答：

按年包考虑，月薪 28～35K，可以结合京东薪资结构和职级再谈。京东常见 16～20 薪。目前已离职，随时到岗。

二、易鑫

6. MicroApp 的原理是什么？和 qiankun 有什么区别 ★★★

- [ ] 已掌握

简短回答：

MicroApp 是京东开源的微前端框架。它用自定义元素 `<micro-app>` 承载子应用，fetch 拉取子应用 HTML，再抽出 JS 和 CSS。JS 放进 Proxy 沙箱，CSS 选择器加上 `micro-app[name=xxx]` 前缀，并拦截 `document` 查询做元素隔离。子应用基本只需要开跨域，不用导出生命周期。qiankun 基于 single-spa，按路由加载，子应用要配合构建插件并接入生命周期，改造成本更高。京东当年就是觉得 qiankun 对存量项目侵入太大，才做了 MicroApp。

展开详解

它解决什么问题

大后台被多个团队维护时，放在一个应用里会越来越大，一个模块出问题会影

响全部。微前端把系统拆成独立开发、独立部署的子应用，运行时再拼起来。

常见方案：iframe 隔离最彻底，但弹窗出不了框架、地址不同步、刷新丢状态、加载慢。single-spa 和 qiankun 按路由加载。MicroApp 像组件一样嵌入。Module Federation 共享的是模块，不是整站。腾讯无界基于 iframe 加 WebComponent。

渲染流程

1. 基座写 `<micro-app name="order" url="http://子应用地址/">`。这是用 CustomElement 注册的标签。
2. 标签插入页面时触发 `connectedCallback`。
3. fetch 子应用 HTML，再 fetch 里面的 script 和 link。**子应用必须允许跨**

域，这是接入时最常见的问题。

4. CSS 做样式隔离，JS 放进沙箱执行，DOM 放进这个标签里。
5. 子应用后来动态创建的 script、link 也会被拦截，做同样处理。

官方说法：用 JS 沙箱、样式隔离、元素隔离、路由隔离模拟 Shadow DOM，再结合 CustomElement，做成类 WebComponent。不用真 Shadow DOM，是因为部分框架（尤其 React）在 Shadow DOM 里跑不起来。

JS 沙箱

默认 with 沙箱：用 Proxy 做一个假 window，再把子应用代码包进 `with(window)` 执行。写全局变量写到代理上，不污染真实 window；代理上没有

的变量，再去真实 window 读。卸载时清理定时器和全局监听。

iframe 沙箱把 JS 放到隐藏的同源 iframe 里跑，隔离更彻底。**Vite 子应用要用 iframe 沙箱**，因为 Vite 开发环境输出的是 ES Module，没法用 with 包起来执行。

局限：有的库会绕过代理拿到真实 window，或者直接改 `document.body`。这时换 iframe 沙箱，或用插件系统在执行前改代码。

样式隔离

```
/* 原来 */
.title { color: red; }
/* 处理后 */
```

```
micro-app[name=order] .title { colo
r: red; }
```

实现上遍历样式表的每条规则，给选择器加前缀。局限：**只能防止子应用影响外面，基座的全局样式仍然会影响子应用。** 所以还要约定样式前缀，或用 CSS Modules。

元素隔离

拦截 `document.querySelector` 等原型方法，子应用只能查到自己容器里的元素。基座和子应用都有 `id="app"` 时，子应用仍能找到自己的根节点。

路由和通信

0.x 用 `baseroute` 分配前缀。1.0 有虚拟路由：默认 search 模式把子应用路由放在地址的 query 里，例如 `?order`

=/detail/1；也可以用 native 模式直接走浏览器地址。

```
// 基座 → 子应用
microApp.setData('order', { token: 'xxx' })
// 子应用接收、回传
window.microApp.addDataListener(data => {})
window.microApp.dispatch({ type: 'logout' })
// 全局
microApp.setGlobalData({ theme: 'dark' })
```

其他：`preFetch` 预加载，`keep-alive` 切走不销毁，生命周期有 created、

beforemount、mounted、unmount、error。

项目里怎么接

基座：

```
import microApp from '@micro-zoe/micro-app'
microApp.start()
```

```
<micro-app name="order" url="http://localhost:3001/" iframe keep-alive></micro-app>
```

子应用主要改跨域：

```
// vite.config.js
export default defineConfig({
  server: { cors: true, headers: {
```

```
'Access-Control-Allow-Origin': '*'
} },
})
```

生产环境在 Nginx 给静态资源加 CORS。判断是否在微前端里用 `window.__MICRO_APP_ENVIRONMENT__`。图片字体用 `window.__MICRO_APP_PUBLIC_PATH__`，避免按基座地址去加载。卸载时执行 `window.unmount`，把 Vue 应用卸掉。

和 qiankun 的区别

	MicroApp	qiankun
加载方式	自定义元素，像组件	基于 single-spa，按路由匹配

	MicroApp	qiankun
子应用改造	基本只开跨域	要接入生命周期。2.x 还要手动改成 UMD；3.x 用官方构建插件，不再手改 UMD
Vite	iframe 沙箱支持较好	需要构建插件
样式	选择器加前缀	3.x 用原生 CSS `@scope`，不支持的浏览器没有降级

qiankun 3.x 的样式隔离依赖浏览器原生 `@scope`，外链样式要能跨域 fetch。旧的 Shadow DOM 严格隔离和实验性选择器改写已经去掉。

追问

为什么不用 iframe？ 弹窗出不了框架，地址不同步，刷新丢状态，通信只能 postMessage，每次都完整加载。

子应用怎么共享 Vue，避免加载多份？ 基座用 CDN 加载公共库，子应用构建时 external，运行时从 window 读。with 沙箱读不到的变量会落到真实 window，所以能读到。也可以用 `globalAssets` 共享资源缓存。代价是版本必须一致，所以很多项目接受重复加载，换取子应用独立。

切换后状态丢了？ 加 `keep-alive`，切走只隐藏不卸载。占用内存，只给常用子应用开。

加载慢？ preFetch、keep-alive、子应用自己分包、CDN 和缓存。

和 Monorepo 各解决什么？ MicroApp 解决运行时拆分和独立部署。

Monorepo 解决开发时共享代码和统一版本。两者没有必然关系，可以只用一个。我们两个一起用。

7. 29 个子应用怎么管？公共依赖怎么共享 ★★

- [ ] 已掌握

简短回答：

`apps` 放基座和子应用，`packages` 放组件、请求和工具。pnpm workspace 用 `workspace:*` 引用，改公共包立刻生效。依赖版本在根目录统一。每个子应用单独启动、构建、部署。我补齐了各环境配置，并把 29 个子应用的地址、技术栈、入口整理进飞书知识库。

8. 为什么用 Monorepo？怎么落地？缺点是什么 ★★

- [ ] 已掌握

简短回答：

多个相关项目放在一个 Git 仓库。好处是公共代码不用发 npm 包、版本统一、跨项目改动一次提交、规范和 CI 统一。缺点是仓库大、全量构建慢、权限不好拆，公共包改坏影响面大。我们用 pnpm workspace：`pnpm-workspace.yaml` 声明 `apps/*` 和 `packages/*`，子应用依赖写成 `"@loan/components": "workspace:*"`。包多了再用 Turborepo，让依赖的包先构建，并缓存没变化的构建结果。

展开详解

目录：

```
loan-admin/
├── pnpm-workspace.yaml
├── apps/main apps/order apps/appro
val
└── packages/components packages/re
quest packages/utils
```

```
packages:
  - 'apps/*'
  - 'packages/*'
```

```
pnpm --filter @loan/order dev
pnpm -r build
pnpm --filter "...[origin/main]" bu
ild   # 只构建相对 main 有变化的包，以
及依赖它们的包
```

Turborepo 里 `"dependsOn": ["^build"]` 表示先构建依赖包，`outputs` 用来做缓存。

为什么常用 pnpm： 依赖存在全局仓库，项目里用硬链接，省磁盘。根 `node_modules` 里只有自己声明的包，不能引用没声明的“幽灵依赖”。npm 和 yarn 把依赖拍平后，代码可能用到别人间接依赖的包，对方升级后你的代码会突然坏。pnpm 原生支持 workspace。

公共包引源码还是先构建： 内部项目常把 `main` 指到源码，由子应用一起编译，改完马上生效。要发布到 npm、或公共包很大时，先打出 dist 再引用。

发版本： Changesets 记录每次变更，发布时算版本号并更新依赖它的包。

CI 不必全量构建： `pnpm --filter` 或 Turborepo 只构建受影响的包，加上构建缓存。

9. 子应用之间怎么跳转？404 和白屏怎么查 ★

- [ ] 已掌握

简短回答：

跳转可以用 `microApp.router.push`，或子应用 `dispatch` 给基座、由基座切路由。404 先看子应用路由 base、history 模式的 Nginx 回退、静态资源 publicPath。白屏先看控制台和网络面板，最常见是子应用没开跨域，MicroApp 用 fetch 拉资源失败；其次是第三方库和沙箱不兼容，可试 iframe 沙箱。

10. 主包从 5MB 降到 2MB：怎么拆的？为什么会有 5MB ★★★

- [ ] 已掌握

简短回答：

先用 `rollup-plugin-visualizer` 看产物。5MB 通常是因为每个 MicroApp 子应用都是独立应用，各自打进一份 Vue、Element Plus、ECharts；公共包如果没有按需导出，会整包打进来；再加上全量引入和 xlsx 这类只在点击时才用的库。我做了拆包、按需引入、路由懒加载，以及把大依赖改成点击时再 `import()`。【说出你们真正拆掉的那一两个大包，以及 5MB 是压缩前还是 Gzip 后。】主包从 5MB 降到 2MB。

展开详解：分包策略和两种工具的写法

为什么一个子应用会到 5MB

1. 微前端里每个子应用独立构建，Vue、组件库、图表库各打一份，不会像单体应用那样全站只留一份。
2. Monorepo 的公共包被引用时，如果没有按需导出，会把所有组件打进这个子应用。
3. `import ElementPlus from 'element-plus'`、`import * as echarts from 'echarts'` 是全量引入。
4. xlsx、富文本、PDF 预览打进了首屏，实际只在某个按钮里用。
5. moment 带全部语言包、图片被内联成 base64、生产环境误开 sourcemap。

Monorepo 只是源码放在一起，**打包时每个应用仍然单独构建**，公共包会进每一个用到它的应用。MicroApp 是运行时再组装，子应用之间的依赖不会自动去重。要么接受重复，要么基座 CDN 加载公共库、子应用 external。部署时还要配好 `base`，并允许跨域。

按什么原则拆

按变化频率和使用范围：框架一个包、UI 库一个包、图表库一个包、其余第三方一个包、多页面共用代码一个包、页面靠路由懒加载自动拆。不要拆得太碎，请求数会上去。

Vite：

```
build: {
  rollupOptions: {
```

```
    output: {
      manualChunks(id) {
        if (!id.includes('node_modu
les')) return
        if (/[\\/](vue|@vue|vue-rou
ter|pinia)[\\/]/.test(id)) return
'vue-vendor'
        if (id.includes('element-pl
us')) return 'element-plus'
        if (id.includes('echarts')
|| id.includes('zrender')) return
'echarts'
        return 'vendor'
      },
    },
  },
}
```

动态 `import()` 会被自动拆出去。Vite 默认不会把第三方库单独拆出，要自己写 `manualChunks`。

Webpack 用 `optimization.splitChunks.cacheGroups`，用 `priority` 决定一个模块归哪组，用 `minChunks` 抽出被多次引用的代码，再用 `runtimeChunk: 'single'` 把运行时代码单独拆出，避免它变化导致所有包的 hash 失效。文件名用 `[contenthash]`。

	Webpack	Vite
规则	`splitChunks`，还有 minSize、maxSize	默认只按动态 import 拆，第三方靠 `manualChunks`
运行时	有独立 runtime Chunk	产物是标准 ES Module，不需要这段运行时

`hash` 是整次构建的哈希，任何文件变了全部文件名都变。`chunkhash` 按包。`contenthash` 按文件内容，最适合长期缓存。

从 5MB 到 2MB 的步骤

1. visualizer 看谁最大。
2. manualChunks 把 Vue、Element Plus、ECharts 拆出去。
3. Element Plus 用 `unplugin-vue-components` 按需；ECharts 从 `echarts/core` 注册用到的图；lodash 换 lodash-es；moment 换 dayjs。
4. xlsx 等改成点击时 `await import('xlsx')`。
5. 公共包用 ES Module 导出，并配置 `sideEffects`，让 Tree Shaking 生效。

6. 关掉生产 sourcemap，压图片，调低小图内联阈值。
7. 再看一次体积和首屏时间。

分析工具：Webpack 用 `webpack-bundle-analyzer`，Vite 用 `rollup-plugin-visualizer`。

Tree Shaking 失效的常见原因：依赖是 CommonJS、Babel 把 import 转成了 require、有副作用却没写 `sideEffects`、用了 `import *` 再动态访问属性。

11. 多环境构建怎么做 ★★

- [ ] 已掌握

简短回答：

Vite 用 `.env.dev`、`.env.test`、`.env.uat`、`.env.production`。变量以 `VITE_`

开头，代码里用 `import.meta.env.VITE_API_BASE`。构建命令是 `vite build --mode test`。后加载的文件覆盖先加载的。原来的问题是【有的子应用缺某个环境、或地址写死】，我把 29 个子应用补齐并统一了变量名。

展开详解

只暴露 `VITE_` 开头的变量，是为了避免把服务端密钥打进浏览器。配置文件里要用变量时：

```
export default defineConfig(({ mode }) => {
  const env = loadEnv(mode, process.cwd())
  return { base: env.VITE_BASE }
})
```

Webpack 用 `dotenv` 读对应 env，再用 `DefinePlugin` 注入；`webpack-merge` 把公共、开发、生产配置拆开。Vue CLI 变量前缀是 `VUE_APP_`。

进阶：一次构建、多环境部署。把地址写进 `public/config.js` 的 `window.__APP_CONFIG__`，`index.html` 先加载它。部署时只换这个文件，测试过的产物就是上线的产物。

12. 发布流程为什么改？两次错误合并是什么 ★★★

- [ ] 已掌握

简短回答：

原来开发分支直接合 main 再发布，要发布的代码和还在开发的代码混在一起，回滚会把正在做的功能一起撤掉。

后来改成上线前拉独立上线分支，验证通过的第二天再合 main。发布和回滚都不碰开发分支。两次拦截是【举例：没测完的功能被合进上线分支 / 冲突处理覆盖了别人的代码】，按旧流程会直接上线。

展开：和 Git Flow、hotfix 的关系

Git Flow 分支多，有 develop、release、hotfix，规范但重。我们接近其中的 release 分支，做了简化。

Trunk Based 是频繁提交主干，靠特性开关和自动化测试，对测试覆盖要求高。

hotfix：从 main 拉分支修复，验证后合回 main，同时合回开发分支，避免下次发布把修复弄丢。

13. 通用 CRUD 组件怎么设计 ★★★

- [ ] 已掌握

简短回答：

配置驱动。搜索项、列、按钮、接口都用一份配置描述，组件内部统一处理请求、分页、loading、重置、删除确认、字典和脱敏。列和搜索项留插槽，请求前后留钩子。某个页面特别特殊时允许不用这个组件，避免组件被塞成无法维护。已经在 6 个模块落地。

展开：配置示例

```
const config = {
  api: { list: getUserList, delete:
deleteUser },
  search: [
```

```
    { field: 'name', label: '姓名',
type: 'input' },
    { field: 'status', label: '状
态', type: 'select', options: statu
sOptions },
  ],
  columns: [
    { prop: 'name', label: '姓名'
},
    { prop: 'phone', label: '手机
号', mask: 'phone' },
  ],
  actions: ['edit', 'delete'],
}
```

14. 脱敏组件怎么做？前端脱敏安全吗 ★★

- [ ] 已掌握

简短回答：

只在前端把手机号显示成 `138****1234` 并不安全，接口里仍是完整号码，打开网络面板就能看到。正确做法是后端返回脱敏数据；查看完整号码走单独接口，做权限校验并记日志。我的组件解决的是展示规则统一：手机号、身份证、银行卡、姓名用同一套规则，以及“点击查看”的交互。【如果当时只做了前端展示，要补一句后来和后端推动了接口脱敏。】

15. Skill 是什么？怎么写、怎么调用 ★★★

- [ ] 已掌握

简短回答：

Skill 是写给 AI 的“某一类任务操作手册”，比如按需求生成 CRUD 页。它是一个目录，里面必须有 `SKILL.md`。开头写 name 和 description，正文写步骤、约定和示例。项目级放 `.cursor/skills/技能名/`，个人级放 `~/.cursor/skills/`。AI 平时只看 description，判断该用时才读全文；也可以在对话里用 `/技能名` 手动调用。我当时是用 create-skill 这个 Skill 来生成的，它会先问用途、存放位置和触发场景，再按规范写出文件。

展开详解

和 Prompt 的区别：Prompt 每次重写；Skill 进仓库、可复用、按需加载。

和 AGENTS.md 的区别：AGENTS.md

每次对话都加载，写全局规矩；Skill 只在做某类任务时加载。

```
.cursor/skills/crud-page/
├── SKILL.md
├── examples.md
└── scripts/
```

```
---
name: crud-page
description: 根据需求生成标准 CRUD 列表页。用户要新建列表页、管理页，或给出字段清单时使用。
---

# 生成 CRUD 列表页

## 步骤
```

```
1. 从需求整理字段：字段名、中文名、类型、是否必填、字典、是否敏感
2. 生成 CrudTable 配置
3. 手机号、身份证必须配置 mask
4. 接口从 @loan/request 引入，不直接写 axios
5. 列出需求里不明确的字段，先确认再改代码
```

description 要同时写“做什么”和“什么时候用”，并带上触发词。正文只写 AI 不知道的团队约定，控制在大约 500 行内，长资料用链接拆出去。给一个已上线页面当示例。

create-skill 是“写 Skill 的 Skill”。面试可以说：我用它按规范生成了业务 Skill，之后 AI 每次生成列表页都会遵守字段映射、脱敏和请求层约定。

判断写得好不好：该用的时候 AI 能自动用上；同类任务结果稳定；AI 犯过的错被补进了 Skill。

16. AGENTS.md、Rules、OpenSpec 是什么？编辑器怎么识别 ★★★

- [ ] 已掌握

简短回答：

AGENTS.md 是仓库里的普通 Markdown，写项目结构、命令和禁止事项。Cursor、Codex 等工具每次对话自动读取，不用手动调用；子目录里的文件在改那个目录时生效。Rules 是 Cursor 自己的，放 `.cursor/rules/`，可以始终生效、按文件类型生效、由 AI 判断，或用 `@规则名` 手动引用。OpenSpec 是先写规格再写代码：

propose 生成提案、规格、方案和任务，人确认后 apply 按任务写代码，archive 把规格合并进长期文档。

展开详解

	何时加载	管什么
AGENTS.md	每次对话自动加载	全局规矩，换工具也能用
Rules	始终 / 按 glob / AI 判断 / 手动	更细，比如只约束 `*.vue`
Skill	做某类任务时	这类任务的步骤
OpenSpec	你发起一次改动时	这次改动的计划、验收和留档

Rules 示例：

```
---
description: Vue 页面规范
globs: apps/**/*.vue
```

```
alwaysApply: false
---


- 列表页使用 CrudTable
- 请求从 @loan/request 引入
- 手机号、身份证使用 MaskText
```

设置里的 User Rules 对你所有项目生效。Claude Code 读的是 `CLAUDE.md`，里面可以写 `@AGENTS.md` 共用一份说明。

OpenSpec 流程

```
npm install -g @fission-ai/openspec
@latest
openspec init
```

init 会生成 `openspec/`，并给当前 AI 工具注册命令。

```
openspec/
├── specs/
系统现在的正式规格
└── changes/add-phone-mask/
    ├── proposal.md
为什么改、改什么
    ├── specs/
这次新增或修改的需求
    ├── design.md
复杂改动才需要
    └── tasks.md
带复选框的任务
```

1. `/opsx:explore`：可选，只讨论，不写文档。

2. `/opsx:propose 改动名`：生成四份文档，**不写代码**。
3. 人按 proposal、specs、tasks 的顺序看。不对就改文件，或让 AI 改。
4. `/opsx:apply`：按 tasks 写代码，做完打勾。中断后重跑，从第一个未完成任务继续。
5. `/opsx:archive`：增量规格合并进 `openspec/specs/`，这次改动移到 `archive/`。

旧版命令是 `/openspec:proposal`、`/openspec:apply`、`/openspec:archive`，流程相同。

规格用“系统应当……”加场景来写，例如：无权限用户打开列表时，手机号脱敏且没有查看按钮。这就是简历里“规格留在 specs、作为后续依据”。

小改动不必走这套流程。中等以上需求多花十几分钟审计划，能少很多返工。规则写了 AI 仍不遵守时，先查 glob 是否匹配、规则是否太模糊，再用 ESLint 和 Code Review 兜底。

17. 同类页面从 4 天缩短到 1 天，怎么算的 ★★★

- [ ] 已掌握

简短回答：

对比的是标准列表加详情加编辑页。【原来：读需求 0.5 天，写骨架 1.5 天，字段、字典、校验 1 天，联调 1 天。】现在 AI 读飞书需求，按 Skill 生成字段映射和骨架，人负责检查、补业务逻辑和联调，大约 1 天。这是几个同类需求的排期对比，业务特别复杂的页

面提升有限。主动说出适用范围，比把数字说满更可信。

18. AI 生成的代码质量怎么保证？出过什么问题 ★★★

- [ ] 已掌握

简短回答：

四层：AGENTS.md 和 Rules 事先写死必须用公共组件、请求层和脱敏；OpenSpec 把需求拆小，一次只生成一块；生成结果和人写的代码一样做 Code Review；规格里有验收条件，做完逐条对。出过的问题是【例如直接用了 axios，没有统一错误处理和 token】。之后把这条写进 Rules，同类错误不再出现。原则是 AI 犯过的错要变成规则。

19. 能现场讲一个 Skill 或 AGENTS.md 吗 ★★

- [ ] 已掌握

简短回答：

准备一份脱敏示例。按这个顺序讲：这个文件解决什么问题、分几部分、举一条规则说明它防住了什么。例如 CRUD Skill 规定敏感字段必须 mask、接口必须从 api 目录引入，并附一个已上线页面。

20. lark-cli 怎么用？需求不清楚怎么办 ★★

- [ ] 已掌握

简短回答：

lark-cli 是飞书命令行，能把文档内容读出来。AI 用它读取需求文档和 bug

文档，再生成开发计划。需求不清楚时，先让 AI 列出不明确的点，拿去问产品，确认后再写代码。这样比以前更早发现需求漏洞。

21. INE 的 AI 报告前端有什么难点 ★

- [ ] 已掌握

简短回答：

【按实际讲。若是流式输出：】接口是 POST 还要带登录头，浏览器自带的 EventSource 不支持，所以用 fetch 的 ReadableStream 一块块读。Markdown 要边收边渲染，并用节流避免卡顿；渲染前用 DOMPurify 防 XSS；离开页面用 AbortController 取消请求。

三、慧策

22. 大数据表格用什么方案？为什么不自己写 ★★★

- [ ] 已掌握

简短回答：

用的是【vxe-table 或其他库名】。Element UI 表格会把所有行都渲染出来，几千行就卡。选型对比过【另外哪些库】，选它是因为支持行列虚拟滚动、固定列、合并单元格，风格接近 Element UI。固定列、动态行高、横向虚拟滚动细节很多，成熟库已经处理过，自己写成本高。我们在它外面做了统一配置封装。

23. 虚拟滚动的原理？下拉框虚拟列表怎么做 ★★★

- [ ] 已掌握

简短回答：

只渲染看得见的行。外层固定高度可滚动，里面用“总行数 × 行高”撑出滚动条，真正的 DOM 只有几十行，再用 `translateY` 挪到正确位置。起始行是 `scrollTop / 行高`，上下多渲染几行当缓冲。下拉框原理相同，但要加上搜索过滤、选中值不在可见区时的回显、打开时滚到选中项、键盘上下选择。Element Plus 有现成的 `el-select-v2`。

展开详解和代码

滚动时：

- 起始索引 = `Math.floor(scrollTop / 行高)`，再减去缓冲行数
- 结束索引 = 起始索引 + 可视行数 + 缓冲
- 偏移 = 起始索引 × 行高
- 用 `transform` 而不是 `top`，因为 transform 不触发回流，还能走合成层

固定行高的核心计算：

```
const start = Math.max(0, Math.floo
r(scrollTop / itemHeight) - buffer)
const end = Math.min(list.length, M
ath.ceil((scrollTop + height) / ite
mHeight) + buffer)
const visible = list.slice(start, e
```

```
nd)
const offset = start * itemHeight
```

模板上：外层监听 scroll；中间一层高度是 `list.length * itemHeight`；内容层 `transform: translateY(offset)`，只 `v-for` visible。

行高不固定： 先按预估高度排位置，渲染后用 ResizeObserver 量真实高度并缓存，维护每项累计位置。滚动时用二分查找定位起始项。总高度 = 已测量项的真实高度 + 未测量项的预估高度。

下拉框额外要做的：

1. 输入防抖后过滤，把滚动条重置到顶部。
2. 用 Map 保存“值 → 选项”，选中值不在当前渲染范围时也能显示文字。

3. 打开时 `scrollTop = 选中索引 * 行高`。
4. 上下键移动高亮，超出可视区就滚动；回车选中。
5. 多选用 Set，判断选中是 O(1)。
6. 十万级选项改成远程搜索，后端分页。

追问：

快速滚动白屏：加大缓冲，滚动更新用 requestAnimationFrame，行内容尽量简单。

和分页、触底加载的区别：分页一次只看一页；触底加载的 DOM 会越积越多；虚拟滚动的 DOM 数量始终固定。可以数据懒加载、渲染用虚拟滚动。

Ctrl+F 和无障碍：没渲染出来的内容，浏览器查找和读屏都看不到，需要的话要自己做搜索。

24. 渲染时间缩短 80%，怎么测的 ★★★

- [ ] 已掌握

简短回答：

用 Performance 面板，同时在数据返回和 `nextTick` 之后用 `performance.now()` 打点。测试数据是【X 行 × X 列】。优化前【X 秒】，优化后【X 毫秒】。这个数字面试前必须自己核实，面试官会追问。

25. 万级单元格一次返回还是分页？前端还做了什么 ★★

- [ ] 已掌握

简短回答：

报表要整体看和汇总，所以【一次返回或分页】。Vue2 会递归给每个字段加 getter，几万行光这一步就慢，只展示的数据用 `Object.freeze`，Vue 会跳过它。Vue3 用 `shallowRef` 或 `markRaw`。汇总和排序放 Web Worker。预处理用 requestAnimationFrame 分批做，避免长时间卡住主线程。

26. 首页白屏怎么定位和优化？全链路优化是什么 ★★★

- [ ] 已掌握

简短回答：

全链路是从输入网址到可以操作的每一步都优化：连接和缓存、资源体积、加载顺序、渲染、运行时、以及监控。白

屏主要是 SPA 要等 JS 下完并执行完才出内容。我用 Lighthouse 和 Performance 看 FCP、LCP，然后做了路由懒加载、组件库和图表按需引入、Gzip、CDN【和骨架屏】。结果是【白屏从 X 秒到 X 秒，体积从 X 到 X】。

展开：指标、配置和追问链路

1. 网络：dns-prefetch、preconnect、CDN、HTTP/2、缓存。
2. 体积：压缩、Tree Shaking、按需、Gzip/Brotli、图片用 WebP。
3. 加载顺序：路由和图片懒加载，关键资源 preload，下一页 prefetch，script 用 defer 或 async。

4. 渲染：骨架屏，减少回流，长列表虚拟滚动，大计算放 Worker。
5. 运行时：防抖节流、避免泄漏、减少无意义的重新渲染。
6. 监控：LCP 建议 2.5 秒内，INP（2024 年起代替 FID）建议 200 毫秒内，CLS 建议 0.1 内。可用 `web-vitals` 采集。

路由懒加载

```
{ path: '/report', component: () => import('@/views/report/index.vue') }
```

Webpack 可用 `/* webpackChunkName: "report" */`。组件级用 `defineAsyncComponent`。

Element Plus 按需

```
import AutoImport from 'unplugin-auto-import/vite'
import Components from 'unplugin-vue-components/vite'
import { ElementPlusResolver } from 'unplugin-vue-components/resolvers'

plugins: [
  AutoImport({ resolvers: [ElementPlusResolver()] }),
  Components({ resolvers: [ElementPlusResolver()] }),
]
```

模板里写 `<el-button>` 就会自动引入组件和样式。Vue2 + Element UI 用 `bab`

`el-plugin-component`。原理是插件扫描模板，只生成用到的 import，打包器就只打这些模块。

ECharts 从 `echarts/core` 引入，再用 `echarts.use` 注册 BarChart、LineChart、Grid、Tooltip 和 CanvasRenderer。lodash 换 lodash-es 并按函数引入。moment 换 dayjs，体积从大约 200KB 降到大约 2KB。

Gzip

构建时用 `vite-plugin-compression` 或 Webpack 的 `compression-webpack-plugin`，大于 10KB 才压。Nginx：

```
gzip on;
gzip_static on;        # 优先用构建好的 .gz，不必每次实时压
```

```
gzip_comp_level 6;
gzip_types text/css application/javascript application/json image/svg+xml;
```

JS/CSS 通常能小 60%～70%。Brotli 还能再小一截，Nginx 要额外模块。图片视频不要再 Gzip。看响应头 `Content-Encoding: gzip` 确认生效。

其他

CDN：`base` 指到 CDN。公共库也可以外链，Webpack 用 externals，代价是依赖 CDN 稳定性。

`preload` 是当前页马上要用，高优先级；`prefetch` 是以后可能用，空闲时低优先级下载。

`defer` 等 HTML 解析完再按顺序执行；`async` 下完就执行，不保证顺序。有依赖用 defer，统计脚本用 async。

首屏排查顺序：HTML 慢、JS 下载慢、JS 执行慢，还是接口慢，再对症处理。中后台一般不必上 SSR。

27. 大文件上传的完整流程 ★★★

- [ ] 已掌握

简短回答：

切片、用 Web Worker 算文件 hash、先问服务端。已有完整文件就是秒传；已有部分分片就只传剩下的，这是断点续传。分片用 FormData 带上 fileHash、编号和文件名，限制 3～6 个并发，失败重试。全部完成后通知服

务端按编号顺序合并，再校验 hash。进度按已传字节除以总大小。暂停用 AbortController，恢复时重新问服务端哪些分片已在。

展开详解和代码

```
const CHUNK_SIZE = 5 * 1024 * 1024
function createChunks(file) {
  const chunks = []
  for (let start = 0; start < file.size; start += CHUNK_SIZE) {
    chunks.push(file.slice(start, start + CHUNK_SIZE))
  }
  return chunks
}
```

`Blob.slice` 只记录范围，不会把整个文件读进内存。

hash 用 spark-md5，放 Worker 里算，否则页面会卡十几秒。算太慢可以用抽样：只取头、尾和中间几段，再加文件大小，重复概率极低。

```
const form = new FormData()
form.append('fileHash', fileHash)
form.append('index', index)
form.append('fileName', fileName)
form.append('chunk', chunk)
```

服务端按编号用流依次写入同一个文件，合并后重算 hash，不一致就要求重传，然后删掉分片。也要清理长期没合并的分片。

注意：分片 2～10MB。并发别超过浏览器对同域名大约 6 个连接的限制，也要考虑服务端。断点以服务端记录为准。文件类型和大小必须后端校验。也可以后端发 STS 临时凭证，前端用 OSS SDK 直传，SDK 自带分片、并发和断点。

追问：

为什么切片：大请求易超时，失败不用整文件重传，还能并发、断点和秒传。

秒传：hash 在服务端已有完整文件，直接成功，本质是按内容去重。

刷新会丢进度吗：不会。同一个文件算出同一个 hash，服务端返回已上传编号，继续传剩下的。

多个文件各自暂停：每个文件一个任务对象，各自保存分片、AbortController、进度和状态。

28. 手写并发控制 ★★

- [ ] 已掌握

简短回答：

启动 limit 个工人，每个工人不断取下一个任务执行，执行完再取，直到取完。JS 单线程，`index++` 不会抢。结果按原下标放回数组。大文件上传就是 `tasks` 为上传函数、limit 为 3～6。失败重试是在单个任务里循环，不是并发函数本身的职责。

展开代码

```
async function runWithLimit(tasks,
limit) {
```

```
  const results = []
  let index = 0
  async function worker() {
    while (index < tasks.length) {
      const current = index++
      try {
        results[current] = await ta
sks[current]()
      } catch (err) {
        results[current] = err
      }
    }
  }
  await Promise.all(Array.from({ le
ngth: Math.min(limit, tasks.length)
}, worker))
```

```
  return results
}
```

29. 进度、暂停、恢复怎么做 ★

- [ ] 已掌握

简短回答：

每个分片用 axios 的 `onUploadProgress` 拿已传字节。总进度是已完成分片大小加正在传的字节，再除以文件大小。每个请求带 AbortController 的 signal，暂停时 abort。恢复就是再调一次“哪些分片已上传”，只传剩余的。

30. 慧经营的数据从哪来？商家用它解决什么 ★★★

- [ ] 已掌握

简短回答：

用户是同时在京东、淘宝、抖音、快手开店的商家。各平台后台分开，费用和结算规则不同，想知道哪个店哪个商品赚钱，只能手工汇总。数据一部分来自旺店通 ERP 的订单、商品和成本，一部分来自各平台开放接口的订单、退款和费用。我们做成订单汇总、记账、销售跟踪、利润和经营分析。利润大致是销售收入减退款、商品成本、平台扣点、运费、推广费和其他费用。难点是各平台口径不同、退款和结算有时间差，所以要支持自定义表项。我负责的三个分析模块占销售额 50% 以上。

31. 利润表自定义公式怎么算 ★★

- [ ] 已掌握

简短回答：

用户通过选择已有指标和运算符组成公式，而不是自由输入。不用 eval。把公式转成后缀表达式再算，或用 expr-eval。指标互相引用时要按依赖顺序算，并检查循环引用。金额不能用 JS 浮点数，按“分”做整数，或用 decimal.js。【说明是前端算还是后端算，前端负责配置和展示。】

32. 埋点怎么做？指令、曝光、怎么保证发出去 ★★

- [ ] 已掌握

简短回答：

手动埋点最准，工作量大；全埋点覆盖全但没有业务含义。我们接了公司鹰眼，页面访问靠路由自动上报，关键行为用 `v-track` 指令。曝光用 IntersectionObserver，进入可视区报

一次再取消监听。上报优先 `navigator.sendBeacon`，页面关闭时也能发出去；失败可退到 1 像素 gif。批量上报，失败的先存 localStorage，下次补发。错误监控是另一套，见第三档。

展开代码

```
const track = {
  mounted(el, binding) {
    el.__trackHandler = () => repor
t(binding.value)
    el.addEventListener('click', e
l.__trackHandler)
  },
  unmounted(el) {
    el.removeEventListener('click',
el.__trackHandler)
```

```
  },
}
```

Vue2 对应 `inserted` 和 `unbind`。用法：`v-track="{ event: 'export_click', page: 'profit' }"`。

停留时长：进入时记时间，`visibility change` 变成隐藏时计算并上报。每条数据带用户、设备、页面、时间、版本。

33. CI/CD 是什么样？你负责哪部分 ★

- [ ] 已掌握

简短回答：

CI 是每次提交自动检查、测试、构建。CD 是产物发布到环境。我们用【GitLab CI 或 Jenkins】：推代码后安

装依赖、ESLint、构建、部署。测试环境自动部署，生产要人工确认。我负责【前端构建脚本、环境变量和部署配置】。前端发布要先传带 hash 的资源，再传 index.html；index.html 不缓存，旧资源先留着，否则正在用旧页面的用户会白屏。回滚就是切回上一版 index.html 或旧镜像，不用重新构建。

展开详解

常用 GitLab CI、GitHub Actions、Jenkins。流水线一般是：检查 → 构建 → 上传产物 → 部署测试 → 人工确认生产 → 通知。

GitLab 里用 stages 分 check、build、deploy。`pnpm install --frozen-lockfile` 保证锁文件一致。`artifacts` 把 dist 交给下一步。生产 job 写 `wh`

`en: manual`。密钥放 CI 变量，不写进仓库。

Monorepo 只构建有改动的子应用，缓存依赖和构建结果，多个子应用并行。

部署方式：rsync 到 Nginx，或上传 OSS+CDN，或打进 Nginx 镜像再部署到 Kubernetes。

白屏的常见发布事故：先更新了 index.html，新 JS 还没传完；部署时删掉了旧 hash 文件；CDN 没刷新。

四、权限、小程序与早期项目

34. 菜单、按钮、路由、数据四级权限怎么做 ★★★

- [ ] 已掌握

简短回答：

菜单：登录后拿权限，没有的不显示。路由：公共路由预先注册，业务路由用 `router.addRoute` 动态加，地址栏直接输入没权限的地址会进 404，`beforeEach` 做登录校验。按钮：`v-permission` 指令，没权限就从 DOM 移除。数据权限主要靠后端过滤，比如只能看自己的店铺，前端只传参和控制展示。前端权限防不住越权，用户可以直接调接口，真正的校验必须在后端。

35. 多版本控制是什么 ★★

- [ ] 已掌握

简短回答：

SaaS 按基础版、专业版卖给租户，每个功能有功能码，每个版本是一组功能

码。用户最终权限 = 租户版本的功能 ∩ 角色权限。运营在后台改配置，不用发版。

36. 小程序为什么从原生改成 uni-app？坑是什么 ★★★

- [ ] 已掌握

简短回答：

原生写法和团队的 Vue 不一样，别人不好维护；uni-app 能复用 Vue 和数据协议，以后还能出其他端。坑包括：平台差异要用条件编译；运行时让包更大，要分包；`setData` 数据量大时卡，不能一次更新整个大对象；ECharts 要换小程序版本。【补一个你真实遇到的坑。】

37. 小程序包大小怎么处理 ★★

- [ ] 已掌握

简短回答：

主包不能超过 2MB，总包不能超过 20MB，以微信当时的规定为准。首页和公共页放主包，功能拆分包；用 `preloadRule` 提前下用户可能去的分包；图片放 CDN；图表用精简版。

38. Android 和 iOS 上，小程序和微信网页有哪些兼容问题 ★★

- [ ] 已掌握

简短回答：

最经典的是 `new Date('2025-01-01 12:00:00')` 在 iOS 上是非法日期，要改成斜杠或用 dayjs。iPhone 底部横条用 `env(safe-area-inset-bottom)`，老系统

再写 `constant`。iOS 键盘收起后页面不回落，blur 时 `scrollTo` 一下。小程序的 canvas、video 层级最高，会盖住弹窗，用同层渲染的 `type="2d"`。微信网页里，iOS 签名要用第一次进入的 URL，Android 用当前 URL。iOS 返回上一页会用缓存，要听 `pageshow` 的 `persisted` 再刷新数据。

展开详解

两端都常见：

- 1px 边框在高清屏偏粗：伪元素加 `transform: scaleY(0.5)`。
- 弹窗滚动穿透：打开时 body 设 `position: fixed` 并记下滚动位置，关闭再恢复。
- iOS 老版本滚动不顺：`-webkit-overflow-scrolling: touch`。

小程序：

- 开发者工具和真机不一致，用真机加 vConsole。
- 键盘用 `adjust-position`、`cursor-spacing`，或 `bindkeyboardheightchange` 拿高度后自己挪布局。
- Android 低端机 `setData` 要只传变化字段。
- 早期 iOS 显示 WebP 要给 image 加 `webp` 属性。
- 静音模式没声音：`obeyMuteSwitch: false`。

微信网页：

- JS-SDK 签名：SPA 用 history 时，iOS 微信记录的是入口 URL。启动时记下入口地址，iOS 签名用它。

- 音视频不能自动播：在 `WeixinJSBridgeReady` 里调用 play。
- 缓存重：index.html 不缓存，资源名带 hash。
- iOS 上非可点击元素的 click 有时不触发，加 `cursor: pointer`。
- `input[type=file]` 的 accept、capture 在不同安卓机上表现不同，要真机测。

调试：vConsole 或 eruda；iOS 用 Mac Safari 远程调试；Android 用 `chrome://inspect`；抓包用 Charles 或 Whistle。

39. “减少 70% 后端开发时间”怎么算？配置协议什么样 ★★★

- [ ] 已掌握

简短回答：

以前每个报表都要单独写接口、逐个对字段。后来约定统一协议：后端返回列定义加行数据，PC 和小程序用同一个组件渲染。新增报表后端只要按协议组织数据。70% 是【重构前后同类报表的后端和联调排期对比，原来 X 天，后来 X 天】。

展开：协议示例

```
{
  "columns": [
    { "key": "shopName", "title": "店铺", "type": "text", "fixed": "left" },
    { "key": "sales", "title": "销售额", "type": "money", "sortable":
```

```
true, "summary": "sum" }
  ],
  "rows": [{ "shopName": "京东旗舰
店", "sales": 128000 }],
  "summary": { "sales": 128000 }
}
```

40. 为什么先用 Superset 再自研？重构了哪 3 个模块 ★★

- [ ] 已掌握

简短回答：

Superset 是开源 BI，后端 Python，前端 React，用来快速验证需求。后来自研是因为【交互和产品风格差、和租户权限不好接、定制成本越来越高】。在它上面改过的 3 个模块是【具体名

称】，这些经验用到了慧数图的设计里。

41. 图表拖拽和联动怎么做 ★

- [ ] 已掌握

简短回答：

仪表盘用网格布局，每个图是可拖拽的格子，位置和图表配置存成 JSON。字段拖到维度、指标后，前端生成查询参数，后端返回数据，再转成 ECharts option。点击某个图的一项，就广播一个筛选条件，其他图带上它重新查。更完整的 BI 和低代码见第三档。

42. 带 2 个人怎么分工？代码不好怎么办 ★★

- [ ] 已掌握

简短回答：

按模块分，每人从需求到上线负责完整模块。我定方案、做公共组件和难点，并做 Code Review。质量不好先看是不熟规范、不熟业务，还是时间太赶。能自动化的交给 ESLint。复杂需求先一起过方案。Review 先看逻辑和边界、安全，再看有没有重复造轮子、命名和性能。

43. 单据状态多了怎么维护 ★★★

- [ ] 已掌握

简短回答：

采购、入库、出库有草稿、待审核、已审核、驳回、作废。不要在每个按钮上写一串 if。用一张表：每个状态允许哪些操作。按钮是否显示 = 当前状态允许 && 用户有权限。新增状态只改配置。

前端只控制展示，并发审核这类校验必须在后端，保证只有一个人成功。这和供应链的采购单、入库单是同一类问题。

展开代码

```
const ORDER_FLOW = {
  draft: { label: '草稿', actions: ['edit', 'submit', 'delete'] },
  pending: { label: '待审核', actions: ['approve', 'reject'] },
  approved: { label: '已审核', actions: ['stockIn', 'void'] },
  rejected: { label: '已驳回', actions: ['edit', 'submit'] },
}
const canDo = (status, action) => O
```

```
RDER_FLOW[status]?.actions.includes
(action)
```

44. 批量录入和商品选择怎么做 ★★

- [ ] 已掌握

简短回答：

商品多时用远程搜索加分页，已选的单独列出。明细用可编辑表格，行多时上虚拟滚动。支持从 Excel 粘贴：听 paste 事件，按制表符拆开填进去。失焦校验单字段，提交前做全表校验并滚到第一处错误。金额小计要防抖。同一商品不能重复属于跨行校验。

45. Android 和内嵌网页怎么通信？有什么安全问题 ★★

- [ ] 已掌握

简短回答：

网页调原生：`addJavascriptInterface` 注入对象，方法要加 `@JavascriptInterface`，4.2 以前没有这个限制，网页能通过反射执行任意代码。也可以让网页跳一个自定义协议地址，原生在 `shouldOverrideUrlLoading` 里拦截。原生调网页用 `evaluateJavascript`。只对白名单域名开放这些方法，敏感能力再单独校验。

46. 做前端负责人时，技术选型看什么 ★

- [ ] 已掌握

简短回答：

团队熟不熟、业务要不要微信能力和原生能力、库的社区和文档能不能长期维

护、小团队能不能复用。所以 Web 用 Vue，App 用原生 Android 加内嵌网页。

47. iBeacon 定位原理 ★★

- [ ] 已掌握

简短回答：

iBeacon 用低功耗蓝牙广播 UUID。手机拿到信号强度 RSSI，按距离衰减公式估算距离。三个已知位置的信标做三边定位，误差用最小二乘法收敛。RSSI 波动大，要做均值或卡尔曼滤波，也可以结合加速度做步行推算。【说你实际用的方案、布了多少个、精度大约多少米。】

展开公式

```
距离 = 10 ^ ((1米处的信号强度 - 当前RSSI) / (10 × 环境衰减系数))
```

环境系数要现场标定。指纹定位是另一种：事先在各点采集信号当指纹，定位时找最像的点。

48. 专利保护什么？你的贡献是什么 ★

- [ ] 已掌握

简短回答：

专利号 ZL 2018 1 0452448.5，保护的是【专利名称和核心方法】。我负责【算法设计和实现 / 是主要发明人】。方案在上海国际会展中心和北京会展中

心落地。面试前把专利名称和权利要求看一遍。

五、Vue 与 React

49. Vue2 和 Vue3 响应式有什么区别 ★★★

- [ ] 已掌握

简短回答：

Vue2 用 `Object.defineProperty` 递归处理每个已有属性。新增、删除属性检测不到，要 `Vue.set`；按下标改数组也检测不到，它是靠重写 push、splice 等 7 个方法。初始化就要递归整棵数据，大数据很慢。Vue3 用 Proxy 代理整个对象，增删改和数组下标都能拦截，嵌套对象只有被访问时才代理。依

赖存在 WeakMap → Map → Set 里：哪个对象的哪个属性，被哪些副作用函数用了。

50. Diff 算法？Vue2 和 Vue3 的区别？key 有什么用 ★★★

- [ ] 已掌握

简短回答：

只比较同一层，标签不同就替换。Vue2 用四个指针做双端比较：旧头对新头、旧尾对新尾、旧头对新尾、旧尾对新头，都不中再用 key 找。Vue3 先从头、从尾跳过相同节点，中间乱序的部分求最长递增子序列，在这个序列里的节点不用移动，从而减少 DOM 操作。Vue3 编译时还会标记动态节点，更新时跳过静态内容。key 用来判断是不是同一个节点。不要用下标当 key，中间

插入时后面的节点会被错误复用，输入框内容会串。

51. nextTick、computed 和 watch、keep-alive ★★

- [ ] 已掌握

简短回答：

改数据不会立刻改 DOM，同一轮多次修改会合并成一次更新。`nextTick` 把回调放到这次 DOM 更新之后。Vue3 用 `Promise.then`，Vue2 按 Promise、MutationObserver、setImmediate、setTimeout 降级。computed 有缓存、必须返回值，适合由已有数据算出新值；watch 没有缓存，适合发请求这类副作用。keep-alive 把组件实例按 key 存起来，切走不销毁，触发

deactivated，回来触发 activated。`max` 超出后按最近最少使用淘汰。

52. Composition API 的好处？Pinia 和 Vuex ★★

- [ ] 已掌握

简短回答：

Options API 把同一个功能拆进 data、methods、watch。Composition API 可以按功能写在一起，并用组合函数复用，替代容易冲突的 mixin，TypeScript 推导也更好。Pinia 没有 mutations，每个 store 天然独立，类型推导好，是官方推荐。Vuex 同步必须走 mutations，模块要处理命名空间。

53. Hooks 为什么不能写在 if 里？两个 Effect 的区别 ★★

- [ ] 已掌握

简短回答：

React 按调用顺序把 Hook 的状态存在 Fiber 上，不是按名字。某次渲染跳过一个 Hook，后面的全都错位。`useEffect` 在浏览器画完之后执行，不挡住显示。`useLayoutEffect` 在 DOM 更新后、绘制前同步执行，用来先量尺寸再调整，避免闪一下。

54. Fiber 是什么 ★

- [ ] 已掌握

简短回答：

React 15 更新时递归整棵树，开始就不能停，大树会卡住输入和动画。Fiber

把组件变成链表上的工作单元，做一小段就看有没有更急的事，例如用户输入，有就先让出主线程。render 阶段算差异，可以中断；commit 阶段改 DOM，不能中断。React 18 的并发特性建立在这上面。

55. Vue 和 React 怎么选 ★★

- [ ] 已掌握

简短回答：

Vue 靠响应式知道哪个组件要更新，模板接近 HTML，官方给了路由和状态库。React 默认重新渲染组件和子组件，要用 memo 等自己优化，写法是 JSX，生态选择多。团队熟悉什么最重要。我主力是 Vue，也有 Superset 的 React 经验，两边都能接。

六、JS、浏览器、工程化、算法

56. 事件循环：看代码说输出 ★★★

- [ ] 已掌握

简短回答：

同步代码先跑完，然后清空全部微任务，包括执行中新产生的微任务，再取一个宏任务，如此重复。微任务有 Promise.then、queueMicrotask、await 后面的代码。宏任务有 setTimeout、事件、整段 script。`new Promise` 里的函数是同步的。

展开例题

```
console.log(1)
setTimeout(() => console.log(2), 0)
Promise.resolve().then(() => consol
e.log(3))
async function fn() {
  console.log(4)
  await null
  console.log(5)
}
fn()
new Promise(resolve => {
  console.log(6)
  resolve()
}).then(() => console.log(7))
console.log(8)
```

输出是 `1 4 6 8 3 5 7 2`。同步是 1、4、6、8；微任务按加入顺序是 3、5、7；最后才是 setTimeout 的 2。面试前在控制台自己跑一遍。

57. 原型链、new、继承 ★★

- [ ] 已掌握

简短回答：

读属性时先看自己，没有就沿 `__proto__` 找，直到 `Object.prototype`，再往上是 null。`实例.__proto__ === 构造函数.prototype`。new 会创建对象、把原型指向构造函数的 prototype、用这个对象当 this 执行构造函数；如果构造函数返回了对象，就用返回值。继承上，ES6 class 是寄生组合继承的语法糖：`Object.create(Parent.prototype)` 加 `Parent.call(this)`。只改原型会共享

引用属性；只 call 继承不到原型方法；两者简单组合会把父构造函数执行两次。

58. Promise/A+ 是什么？手写 Promise、all、race ★★★

- [ ] 已掌握

简短回答：

Promise/A+ 是社区标准，只规定 `then` 怎么工作，让不同实现能互相配合。它没有规定构造函数、catch、all，那些是 ES6 加的。核心是三种状态且只能变一次；then 的回调必须异步执行、最多一次；then 返回新 Promise。回调返回普通值，新 Promise 就成功；抛错就失败；返回另一个 Promise，就等它的结果。返回自己要报 TypeError，防止循环。对象只要有

then 方法，就按 Promise 处理，这是为了兼容别的库。

展开：解决过程和 all / race

处理返回值 x 时：x 就是新 Promise 自己，直接拒绝。x 是对象或函数，就取 `x.then`；then 是函数就调用它，成功和失败回调只认第一次；then 不是函数，就把 x 当成功值。读取 then 时抛错，就用这个错拒绝。

```
function promiseAll(promises) {
  return new Promise((resolve, reje
ct) => {
    const list = Array.from(promise
s)
    const results = []
    let count = 0
```

```
    if (list.length === 0) return r
esolve(results)
    list.forEach((p, i) => {
      Promise.resolve(p).then(value
=> {
        results[i] = value
        if (++count === list.lengt
h) resolve(results)
      }, reject)
    })
  })
}

function promiseRace(promises) {
  return new Promise((resolve, reje
ct) => {
    for (const p of promises) Promi
se.resolve(p).then(resolve, reject)
```

```
  })
}
```

all 全部成功才成功，结果按原顺序；一个失败就失败。race 谁先有结果用谁的。allSettled 等全部结束并保留每个的状态。any 有一个成功就成功。

then 不传函数时，值会继续往后传，所以 `.then().then(v => v)` 仍能拿到值。回调必须异步，是为了无论 Promise 是否已经完成，回调都在当前同步代码之后，行为一致。官方测试套件是 `promises-aplus-tests`。

59. 手写防抖、节流、深拷贝、LRU ★★

- [ ] 已掌握

简短回答：

防抖是连续触发只执行最后一次，适合搜索框。节流是一段时间内最多一次，适合滚动。深拷贝要处理循环引用、日期和正则；`JSON.parse(JSON.stringify())` 会丢掉函数、undefined，循环引用会报错。现代浏览器可用 `structuredClone`，但它不拷函数。LRU 用 Map 的插入顺序：访问时删掉再插入，使它变成最新；满了就删 Map 的第一个键。keep-alive 的 max 也是这个思路。

展开代码

```
function debounce(fn, delay) {
  let timer = null
  return function (...args) {
    clearTimeout(timer)
    timer = setTimeout(() => fn.app
```

```
ly(this, args), delay)
  }
}

function throttle(fn, interval) {
  let last = 0
  return function (...args) {
    const now = Date.now()
    if (now - last >= interval) {
      last = now
      fn.apply(this, args)
    }
  }
}

function deepClone(value, map = new
WeakMap()) {
  if (value === null || typeof valu
```

```
e !== 'object') return value
  if (value instanceof Date) return
new Date(value)
  if (value instanceof RegExp) retu
rn new RegExp(value)
  if (map.has(value)) return map.ge
t(value)
  const copy = Array.isArray(value)
? [] : {}
  map.set(value, copy)
  for (const key of Reflect.ownKeys
(value)) copy[key] = deepClone(valu
e[key], map)
  return copy
}

class LRUCache {
  constructor(capacity) {
```

```
    this.capacity = capacity
    this.map = new Map()
  }
  get(key) {
    if (!this.map.has(key)) return
-1
    const value = this.map.get(key)
    this.map.delete(key)
    this.map.set(key, value)
    return value
  }
  put(key, value) {
    if (this.map.has(key)) this.ma
p.delete(key)
    this.map.set(key, value)
    if (this.map.size > this.capaci
ty) this.map.delete(this.map.keys
().next().value)
```

```
  }
}
```

60. 渲染流程、回流和重绘 ★★★

- [ ] 已掌握

简短回答：

HTML 成 DOM，CSS 成 CSSOM，合成渲染树，再布局、绘制、合成。`display: none` 不进渲染树。回流是位置或尺寸变了，要重新算布局；改颜色只是重绘。回流一定会重绘。减少回流：用 class 批量改样式，用 DocumentFragment 批量改 DOM，循环里不要交替读写布局属性，动画用 transform 和 opacity，长列表用虚拟滚动。

61. 强缓存和协商缓存 ★★★

- [ ] 已掌握

简短回答：

强缓存有效期内不发请求。`Cache-Control: max-age` 优先于 `Expires`。`no-cache` 是可以存但每次要问服务器，`no-store` 是完全不存。过期后走协商缓存：带 ETag，没变就 304。ETag 比 Last-Modified 精确，因为修改时间只到秒，而且内容没变、时间变了也会被当成更新。实践上 index.html 用 no-cache，带 hash 的 JS、CSS 用很长的 max-age。

62. HTTPS、HTTP/2、TCP 握手 ★★

- [ ] 已掌握

简短回答：

HTTPS 用非对称加密协商出会话密钥，再用对称加密传数据，因为对称加密快。TLS 1.3 把握手缩到一个往返。HTTP/2 一个连接上可以同时传多个请求，解决了 HTTP/1.1 应用层的队头阻塞，还有头部压缩。但 TCP 丢包时所有流仍要等重传，HTTP/3 改用基于 UDP 的 QUIC。三次握手是为了确认双方收发都正常，并挡住网络里滞留的旧连接请求。四次挥手是因为两个方向要分别关；客户端发完最后的 ACK 要等 2MSL，确保对方收到，也让旧报文消失。

63. 跨域、XSS、CSRF ★

- [ ] 已掌握

简短回答：

跨域是协议、域名、端口有一个不同。主流是服务端返回 CORS 头；开发用代理，生产用 Nginx。XSS 是把脚本注入页面。Vue 和 React 默认转义，慎用 v-html，必须用时先 DOMPurify，Cookie 加 HttpOnly。CSRF 是别的网站凭用户已登录的 Cookie 发请求。Cookie 设 SameSite，或用 CSRF Token，或校验 Origin。用 Authorization 头传 token、不用 Cookie，就不受 CSRF 影响。

64. Vite 为什么快？和 Webpack 的区别 ★★★

- [ ] 已掌握

简短回答：

Webpack 开发时要先把整个项目打包再启动，项目越大越慢。Vite 利用浏览

器原生 ES Module，请求哪个文件才编译哪个，启动时间和项目大小基本无关。第三方依赖用 Go 写的 esbuild 预先打包并缓存。改一个文件，热更新只让这个模块失效。生产环境 Vite 仍要打包，因为大量小文件请求在线上性能差，用的是 Rollup，新版本在换成 Rolldown。分包的具体写法和 5MB 到 2MB，见第 10 题。

展开：loader、plugin、Tree Shaking

loader 只转换某一类文件，例如 babel-loader、css-loader，执行顺序从右到左。plugin 监听构建过程的钩子，能做生成 HTML、抽 CSS、注入变量。

Tree Shaking 能删未使用的导出，是因为 ES Module 的 import 必须写在顶层，编译期就知道谁被用了。CommonJS 的 require 是运行时的，做不到。打包器先标记，压缩器再删除。有的文件被引入就会执行副作用，要在 package.json 的 `sideEffects` 里声明，否则可能被误删。Babel 不要把 import 转成 require，否则 Tree Shaking 失效。

65. TypeScript 泛型和常用工具类型 ★

- [ ] 已掌握

简短回答：

泛型是把类型当参数，调用时再指定。例如 `request<User[]>` 让返回值里的 data 是用户数组。约束写成 `T extends`

`{ length: number }`。Partial 全可选，Required 全必填，Pick 挑几个属性，Omit 去掉几个，Record 构造键值类型，ReturnType 取函数返回值。Partial 的实现是映射类型：`{ [K in keyof T]?: T[K] }`。

66. 算法准备和数组转树 ★★

- [ ] 已掌握

简短回答：

京东前端通常 1～2 道中等题。优先：两数之和、无重复最长子串、有效括号、反转链表、二叉树遍历、爬楼梯、最大子数组和。和权限菜单直接相关的是数组转树：先用 Map 存 id 到节点，再按 parentId 挂到父节点的 children，找不到父节点的就是根。

展开代码

```
function arrayToTree(list, rootId =
0) {
  const map = new Map()
  const roots = []
  list.forEach(item => map.set(ite
m.id, { ...item, children: [] }))
  map.forEach(node => {
    const parent = map.get(node.par
entId)
    if (node.parentId === rootId ||
!parent) roots.push(node)
    else parent.children.push(node)
  })
  return roots
}
```

先讲思路和复杂度，写完用一个例子走一遍。

七、扩展专题

这些简历上点到过，或面试官可能顺口一问。每个能讲清下面的简短回答即可，详解留着被追问时再看。

67. A/B 测试怎么做 ★

- [ ] 已掌握

简短回答：

同一个功能做两个版本，把用户分成组，比点击率或转化率。分流不要用随机数，要对“实验名 + 用户 ID”做 hash，再对 100 取模，这样同一个人每次都在同一组。埋点要报曝光和转化，并带上实验名和版本。页面先渲染

默认版、配置回来后再切换，会造成闪烁，所以要尽早拿配置，或先显示骨架屏。和灰度的区别：灰度看稳定性，A/B 看哪个版本效果好。

展开详解

流程是假设、定指标、分流、上线、看统计显著性、决定全量或下线。除了核心指标，还要有一个不能变差的护栏指标。

```
function getBucket(userId, experimentKey) {
  return murmurhash(`${experimentKey}:${userId}`) % 100
}
```

未登录用设备 ID。AA 测试是两组看完全相同的版本，用来检查分流系统本身

有没有偏差。多个实验用分层，让不同层的分流互不影响。样本量不够、没覆盖工作日和周末，不要下结论。前端要保证埋点不重不漏，显著性一般由数据平台算。

68. 大屏怎么做？会遇到什么问题 ★

- [ ] 已掌握

简短回答：

设计稿一般是 1920×1080。最常用的适配是按这个尺寸写死，再用 scale 按屏幕缩放，比例不变，但比例不同会留黑边。图表用 ECharts，实时数据用 WebSocket。窗口变化要 `chart.resize()`。数据大时开 large、降采样，只更新数据不重建图表。大屏常年不关，定

时器、图表实例和 WebSocket 数据要清理，否则内存一直涨。

展开详解

```
function fit() {
  const scale = Math.min(innerWidth / 1920, innerHeight / 1080)
  screenEl.style.transform = `scale(${scale}) translate(-50%, -50%)`
}
```

vw/vh 能铺满但会变形。rem 按屏宽改根字号。实际常是整体 scale，背景用 vw。

其他问题：scale 太小字会糊，给 ECharts 设 devicePixelRatio。断线要退避重连和心跳。地图 GeoJSON 很大，要压缩并先出 loading。Canvas

适合数据多、频繁重绘；SVG 适合元素少、要单独绑事件。数据几分钟才变一次时，轮询比 WebSocket 简单；只要服务端推、浏览器不发，也可以用 SSE。

内存泄漏用 Memory 面板拍两次堆快照对比。

69. BI 是什么？动态图表和低代码怎么做 ★

- [ ] 已掌握

简短回答：

BI 是让业务人员不写 SQL，用拖拽做报表。核心是数据源、数据集、图表、仪表盘和权限。维度是按什么分组，例如店铺、日期；度量是统计什么，例如销售额求和。动态图表是配置驱动：用

户拖拽生成一份 JSON，后端按它生成 SQL，前端用对应图表的适配器转成 ECharts option。低代码也是同一思想：用 JSON 描述页面，渲染器负责画出来，设计器只是编辑这份 JSON 的工具。我做的表格配置化和 CRUD 配置，就是低代码里性价比最高的一部分，并不是一个完整的拖拽平台。

展开详解

自研 BI 要研究什么

竞品有 Tableau、Quick BI、FineBI、Superset、DataEase。要定数据源、用 ClickHouse 或 Doris 这类分析库、查询缓存、维度和度量、计算字段、图表插件、筛选联动和钻取、行级和列级权限、多租户隔离、导出和定时推送。

查询慢时：后端预聚合和缓存，前端防重复请求、允许取消、限制返回条数。

图表配置示例：type、datasetId、dimensions、measures、filters、sort。联动是点击后广播“字段 = 值”，其他图带上重新查。钻取是从年点到月，路径上逐级加筛选。布局用 vue-grid-layout，整份仪表盘存成 JSON。

低代码的组成

```
物料（组件 + 每个属性怎么配）
  → 设计器（画布、属性面板、大纲、撤销）
  → Schema（JSON）
  → 渲染器，或生成源代码
```

一个节点有组件名、属性、子节点、事件。`{{ }}` 用来绑定变量。属性面板是根据物料元信息动态生成的表单。渲染

器递归 Schema，用组件名找到组件，Vue 里用 `h()`。

设计器拖拽可用 HTML5 拖拽或 sortablejs。画布常用 iframe，避免设计器样式和组件样式互相污染。撤销可以每次存一份 Schema 快照，或记录操作和它的反向操作。

发布有两种：线上直接渲染 Schema，改完即生效；或把 Schema 生成 Vue 源码，之后还能手改，性能也更好。

从零做的顺序：先限定只做表单和列表，再定 Schema，然后渲染器、物料、设计器、数据源和事件、保存发布。可参考的开源有阿里 LowCodeEngine、百度 amis、formily。

局限：复杂交互很难用配置表达，配置太复杂后学习成本可能超过写代码。所以要允许复杂的地方写代码。

70. 错误监控怎么做 ★

- [ ] 已掌握

简短回答：

JS 错误听 `error`。资源加载失败的 error 不冒泡，必须在捕获阶段听。没人 catch 的 Promise 听 `unhandledrejection`。Vue 组件错误要配 `app.config.errorHandler`，它不会冒泡到 window。接口错误放在 axios 拦截器。跨域脚本只显示 Script error 时，script 加 `crossorigin`，CDN 返回 CORS 头。压缩代码对不上源码时，sourcemap 上传到监控平台，不要部署到线上。

展开详解

```
window.addEventListener('error', ev
ent => {
  const target = event.target
  if (target && (target.src || targ
et.href)) {
    report({ type: 'resource', url:
target.src || target.href })
  } else {
    report({ type: 'js', message: e
vent.message, stack: event.error?.s
tack, line: event.lineno })
  }
}, true)
```

白屏：加载完成后在屏幕上取几个点，`document.elementsFromPoint` 如果拿到

的都是 html、body 或根节点，就认为没渲染出来。

用户行为回溯：记录报错前的点击、路由和接口，这叫面包屑。rrweb 可以录制回放。相同错误要聚合，突然变多时用飞书机器人告警。平台可以用 Sentry，或公司自研。

埋点数据太大时：批量、采样、合并相同事件。上报方式见第 32 题。

71. 团队怎么推动 AI 落地 ★

- [ ] 已掌握

简短回答：

先解决能不能用：统一工具，确认代码会不会被拿去训练，密钥和用户数据不能发给模型。再小范围试点最适合的场景，例如 CRUD、类型、单测，用前后

耗时说服人。然后把项目知识写成 AGENTS.md、Rules 和 Skill，并用 MCP 接飞书、接口文档。最后嵌进需求、编码、Review、排查的流程。看交付周期和 bug 率，不要只看 AI 写了多少行。我的做法是先在自己模块试点，用 4 天到 1 天的数据做分享。

展开：风险

质量靠人 Review、ESLint 和测试。AI 编造不存在的 API 时，要给真实文件当上下文，生成后验证。新人必须能解释自己提交的代码。适合规则清楚的重复工作；核心业务决策和安全敏感代码由人主导，AI 只辅助。

MCP 是让 AI 调用外部工具的协议，例如读飞书文档、读接口定义、读设计稿。

72. 供应链有哪些环节？采购单页面怎么设计 ★★

- [ ] 已掌握

简短回答：

计划、采购、入库、库存、销售占库存、出库履约、退货、结算。对应采购、仓储、库存、订单、运输、结算这些系统。前端特点是表格和表单复杂、状态多、权限细、数据大、不能算错。采购单页面可以直接套我做过的东西：列表用 CRUD 配置；状态用状态机；明细支持远程选商品、粘贴 Excel，金额用整数分计算；提交防重复；按钮和数据按权限控制。

展开：页面设计

1. 列表：单号、供应商、状态、时间；操作按状态和权限显示。

2. 状态：草稿、待审核、已审核、部分入库、全部入库、驳回、作废。
3. 新建：主表加商品明细，远程搜索、批量录入、粘贴导入，提交前校验并定位第一处错误。
4. 详情：单据、明细、审批记录。
5. 明细行多时虚拟滚动，列表分页，导出单独做。
6. 提交按钮 loading，后端接口幂等。
7. 采购员只能看自己负责的供应商，这是数据权限，过滤在后端。

八、反问

准备 2～3 个：

- 团队做的是采购、库存还是履约？用户是内部运营还是商家？

- 前端是 Vue 还是 React？有没有微前端或低代码平台？
- 团队现在的 AI 提效做到哪一步？
- 入职三个月最希望我解决什么？

面试前最后检查

- [ ] 所有【】换成真实数据和例子
- [ ] 主包 5MB 是压缩前还是 Gzip 后，拆掉的最大包叫什么
- [ ] 大数据表格的库名、优化前后的秒数
- [ ] 两次错误合并各是什么
- [ ] 离职原因和半年空档能自然说出来
- [ ] 第 28、56、58、59 题的代码自己手写过，不只看过

- [ ] 准备一份脱敏的 Skill 或 AGENTS.md，手机里能打开